

Notebook

February 8, 2026

1 Кривые Монтгомери и Небезопасные кручения

В предыдущих упражнениях мы рассмотрели преимущества эллиптических кривых, но мы остановились только на короткой форме Вейерштрасса уравнения кривой. Тому есть две причины:

- она легче для понимания
- проще вывести операции на группе

Однако, существуют другие уравнения, описывающие специфические семейства эллиптических кривых, которые могут быть немного труднее для понимания, но у которых есть безусловные достоинства. Одно из таких семейств - это **кривые Монтгомери**. Они лишь одно из замечательных открытий доктора наук Питера Лоуренса Монтгомери, который сделал крупный вклад в современную криптографию. К сожалению, в 2020м году он ушёл из жизни. Вы можете прочитать его некролог, в котором упоминаются его достижения, здесь: [Peter Lawrence Montgomery](#)

1.1 Кривые Монтгомери

Кривые Монтгомери определяются уравнением над полем F : $M_{A,B} : By^2 = x^3 + Ax^2 + x$, где $A, B \in F$ и $B(A^2 - 4) \neq 0$. Мы можем преобразовать любую кривую Монтгомери в короткую форму Вейерштрасса. Для этого надо сначала разделить каждый член уравнения на B^3 и заменить переменные x и y на $u = \frac{x}{B}$ и $v = \frac{y}{B}$ соответственно. В результате получится выражение:

$$v^2 = u^3 + \frac{A}{B}u^2 + \frac{1}{B^2}u$$

Далее можно преобразовать его в короткую форму, заменив u на $t - \frac{A}{3B}$:

$$v^2 = \left(t - \frac{A}{3B}\right)^3 + \frac{A}{B}\left(t - \frac{A}{3B}\right)^2 + \frac{1}{B^2}\left(t - \frac{A}{3B}\right)$$

которое превращается в

$$v^2 = t^3 + \left(\frac{3 - A^2}{3B^2}\right)t + \left(\frac{2A^3 - 9A}{27B^3}\right)$$

Таким образом, преобразование координат следующее:

$$(x, y) \mapsto (t, v) = \left(\frac{x}{B} + \frac{A}{3B}, \frac{y}{B}\right), a = \frac{3 - A^2}{3B^2}, b = \frac{2A^3 - 9A}{27B^3}$$

Хоть мы можем выразить любую кривую в форме Вейерштрасса, не всем кривым суждено быть кривыми Монтгомери. Кривые могут быть выражены в форме Монтгомери т.т.т.к (тогда и только тогда, когда) у изначальной кривой $E_{a,b}$ есть данные свойства: 1. Порядок $E_{a,b}$ делится на 4 2. У $z^3 + az + b = 0$ есть хоть один корень $\alpha \in F$, где F - это поле, на котором определена кривая 3. $3\alpha^2 + a$ - квадратичный остаток в F

Итак, кривые Монтгомери - подмножество эллиптических кривых. В чём же их преимущество? Вот в чём: умножение точки, принадлежащей кривой Монтгомери, на скаляр выполняется при помощи эффективного алгоритма, называющегося лестница Монтгомери:

```
function ladder(u, k):
    u2, w2 := (1, 0)
    u3, w3 := (u, 1)
    for i in reverse(range(bitlen(p))):
        b := 1 & (k >> i)
        u2, u3 := cswap(u2, u3, b)
        w2, w3 := cswap(w2, w3, b)
        u3, w3 := ((u2*u3 - w2*w3)^2,
                  u * (u2*w3 - w2*u3)^2)
        u2, w2 := ((u2^2 - w2^2)^2,
                  4*u2*w2 * (u2^2 + A*u2*w2 + w2^2))
        u2, u3 := cswap(u2, u3, b)
        w2, w3 := cswap(w2, w3, b)
    return u2 * w2^(p-2)
```

Cryptopals считают, что вам не нужно понимать, почему и как это работает. В целом, я согласен. Но если вам захочется понять математику и идеи, породившие этот короткий но удивительный алгоритм, прочитайте [Montgomery Curves and Montgomery ladder](#). Там также можно почерпнуть много нового по теории эллиптических кривых.

Но всё же стоит рассмотреть некоторые особенности этого алгоритма:

- Лестница Монтгомери - это лестница по одной координате. Это значит, что лестница вычисляет u точки $k * P$, имея лишь координату u точки P . Поскольку используется только координата u , и (u, v) и $(u, -v)$ выражены одним значением u .
- Переменная w , применяемая в алгоритме, это координата в альтернативном представлении точки. Точки на эллиптических кривых могут быть представлены в так называемых проективных координатах. Например, короткой формы Вейерштрасса $y^2 = x^3 + ax + b$ соответствует уравнение с проективными координатами $Y^2Z = X^3 + aXZ^2 + bZ^3$, так что координаты - $(X : Y : Z)$. Также точки $(X : Y : Z)$ и $(\lambda X : \lambda Y : \lambda Z)$ эквивалентны, если $\lambda \neq 0$. И всегда можно вычислить эквивалентную пару $(x : y)$ как $(X/Z : Y/Z)$ (если $Z \neq 0$). Точка на бесконечности обычно обозначается $(0 : 1 : 0)$. Проективные координаты могут повысить эффективность вычислений. Деление (или вычисление обратного числа) использует много ресурсов. Различные источники дают примерную

оценку в 80~100 умножений, поэтому многократное использование деления нежелательно. Лестница Монтгомери использует проективные координаты, чтобы отложить операцию деления до последнего момента. Вы можете помнить, что $a^{(p-2)} = a^{-1} \bmod p$, т.е. мы вычисляем аффинную координату (стандартное представление) u прямо перед возвратом из функции.

- $\text{cswap}(a,b,c)$ обозначает взаимную замену a и b , если c . Эта операция может быть имплементирована без ветвления:

```
t := a xor b
m := neg c
t := t and m
a := a xor t
b := b xor t
```

Представьте, что это исполняется на 32-битной архитектуре. Поскольку значения, которые мы используем при операциях над точками, скорее всего не влезут в 32 бита, мы манипулируем указателями на структуры в памяти. Таким образом a и b - указатели. Сначала мы вычисляем хог (исключающее или) a и b , далее мы вычисляем отрицание c . Если c равно 0, то m тоже 0. Если c равно 1, то m становится $0xffffffff$. Далее мы вычисляем логическое И от t и m . Таким образом t либо останется неизменным (если m равно $0xffffffff$) или станет 0. Далее мы применяем ксор к a и b (по отдельности, а не друг с другом) с либо 0 (что не меняет их значения), либо $(a \text{ хог } b)$, которое меняет их местами. Почему так важно избегать конструкций ветвления при операциях с закрытым ключом? Конструкции ветвления особо уязвимы к атакам на побочные каналы, поскольку исполняются различные пути и инструкции. Такие атаки могут использовать побочные каналы по времени, энергопотреблению или ЭМИ. cswap лучше защищён от них.

- Алгоритм использует так называемое дифференциальное сложение. Мы можем сложить две точки P и Q , только если мы знаем разность между ними. Алгоритм умно использует сложение и удвоение, сохраняя разность между двумя значениями, пока он вычисляет результат
- Алгоритм правильно обрабатывает особые случаи (добавление точки к точке на бесконечности и т.п.) без ветвления.

1.2 Кручение

Помните, как мы использовали “ядовитые” точки, чтобы заставить алгоритмы сложения и удваивания точек для короткой формы Вейерштрасса выдать нам секретный ключ? Поскольку сейчас можно манипулировать лишь одной координатой, тот трюк не сработает. Но ещё не всё потеряно.

Когда мы искали точки на кривых с удобно факторизуемыми порядками, мы проверяли, что для данного x полученное y^2 было квадратичным остатком в F . В конце концов, если это было не так, мы не могли вычислить квадратичные корни. Смотрите внимательно. Представьте, что у нас есть обычная эллиптическая

кривая в короткой форме Вейерштрасса, определённая на поле F :

$$y^2 = x^3 + ax + b$$

Что произойдёт, если мы умножим левую часть уравнения на $d \neq 0$?

$$dy^2 = x^3 + ax + b$$

Если d - квадратичный остаток и некоторое m - один из его корней, то мы можем заменить $my \mapsto y'$ и получить:

$$y'^2 = x^3 + ax + b$$

То же уравнение, по сути та же кривая. Поменяются лишь позиции точек, но не отношения между ними. А что произойдёт, если d - неквадратичный остаток? Все те x , которые раньше порождали точки, т.к. y^2 был квадратичным остатком, теперь не порождают точки на кривой. Теперь это делают все остальные значения x . Такая кривая называется **квадратичным кручением** оригинальной кривой. Таким образом все значения x , которые превращают правую половину равенства в неквадратичный остаток, порождают другую кривую. Как вы можете представить, то же самое справедливо и для кривой Монтгомери, поскольку левая часть равенства - v^2 . И если мы применим лестницу Монтгомери к значению u , которое создаёт неквадратичный остаток в правой части равенства, мы вычислим координату u точки на квадратичном кручении. Теперь мы можем снова передавать оракулу точки на альтернативной кривой, но чтобы это имело для злоумышленника пользу, надо знать порядок этой кривой. Так давайте его вычислим:

1. Мы знаем, что есть p различных элементов поля F_p . Один из них особый - 0, т.е. есть $p - 1$ обычных значений и 0.
2. Правая часть уравнения принимает все значения в F_p . Если она равна 0, то точка существует и на оригинальной кривой и на квадратичном кручении. Для всех остальных значений в F_p правая часть равенства либо квадратичный остаток, и на оригинальной кривой появляется две точки, либо неквадратичный остаток, и на квадратичном кручении появляется две точки.
3. У обеих кривых есть точка на бесконечности. Таким образом общее количество точек на двух кривых равно $2 + 2(p - 1) + 2 = 2(p + 1)$. Мы можем вычислить количество точек на квадратичном кручении, вычтя порядок оригинальной кривой из $2(p + 1)$.

Также стоит знать, что порядок изначальной кривой равен $\#E(F) = p + 1 - t$, где t называется след Фробениуса и в соответствии с теоремой Гассе находится в следующих пределах $|t| \leq 2\sqrt{p}$.

1.3 Task

Мы позаимствуем параметры кривой у Cryptopals (поскольку они великолепны для обучения). Изначальная короткая форма Вейерштрасса:

$y^2 = x^3 - 95051x + 11279326$ над $GF(233970423115425145524320034830162017933)$ Один из возможных генераторов - $(182, 85518893674295321206118380980485522083)$. Порядок кривой

$$233970423115425145498902418297807005944 = 2^3 * 29246302889428143187362802287225875743$$

Преобразование точки в точку на кривой Монтгомери крайне просто (снова, огромное спасибо Cryptopals):

$$(u, v) = (x - 178, y)$$

Форма Монтгомери

$$v^2 = u^3 + 534u^2 + u$$

Секретный ключ сервера $k = 0 \bmod 1571528514013$. В принципе, решить проблему дискретного логарифма в Python в этой подгруппе можно, но это требует слишком много времени (у меня это заняло несколько часов в одном потоке). Но всё же вам придется использовать Полларда. Его можно использовать и с Монтгомери. Используйте вариацию ро.

```
//Один прыжок:  
j:=hash(point)  
xT=(xT*j)%subgroup_order  
yT=ladder(yT,j)
```

Подсказка: j не должен принимать значение 0 или кратное порядку подгруппы, поскольку результат лестницы Монтгомери зафиксирован в 0.

Ваша цель - снова найти закрытый ключ сервера. Чтобы этого добиться:

1. Вычислите порядок квадратичного кручения
2. Факторизуйте его в $(4, 11, 107, 197, 1621, 105143, 405373, 2323367, 1571528514013)$
3. Найдите точку на квадратичном кручении (просто найдите u , которая превратит правую сторону уравнения в неквадратичный остаток)
4. Отправьте точки из подгрупп квадратичного кручения (то же, что и в предыдущем задании)
5. КТО, но вам придётся это делать постепенно. Каждое значение u представляет 2 точки, так что когда найдёте какое-то r для подгруппы порядка d , на самом деле потенциальных остатков два: r и $d - r$. Если бы вы собрали все остатки независимо, вам бы пришлось попробовать все возможные комбинации. Это необоснованно трудно. Допустим, у вас есть (r_1, d_1) и (r_2, d_2) . Вы можете вычислить 4 возможных остатка по модулю $d_1 d_2$, далее проверить, какие из этих 4х применимы, отправив точки, введенные в подгруппу порядка $d_1 d_2$. Вы снизите количество возможностей до 2. Повторите процесс для d_i , где $i > 2$
6. Profit

P.S. Если вы хотите ещё узнать об эллиптических кривых или вас интересуют стандартизированные кривые, [Safecurves](#) - хорошее начало. Если вы хотите имплементировать кривую, например для учёбы (пожалуйста, **НЕ РЕАЛИЗУЙТЕ СВОЮ КРИПТОГРАФИЮ**) посетите [Explicit Formulas Database](#).

```
[ ]: def create_ladder(p,A):
    bitlen_p=0
    p1=p
    while p1!=0:
        p1>>=1
        bitlen_p+=1

    def ladder(u,k):
        nonlocal p,bitlen_p,A
        (u2,w2)=(1,0)
        (u3,w3)=(u,1)
        for i in range(bitlen_p):
            b=(k>>(bitlen_p-1-i))&1
            if b==1:
                u2,u3,w2,w3=u3,u2,w3,w2
                u3,w3=pow((u2*u3-w2*w3)%p,2,p) , (u*pow(u2*w3-w2*u3,2,p))%p
                u2,w2= pow(pow(u2,2,p)-pow(w2,2,p),2,p) , u
                ↪ (4*u2*(w2*(pow(u2,2,p)+A*((u2*w2)%p)+pow(w2,2,p))%p)%p)
            if b==1:
                u2,u3,w2,w3=u3,u2,w3,w2
        return (u2*pow(w2,p-2,p))%p
    return ladder
```

```
[ ]: import socket
import re
class VulnServerClient:
    def __init__(self,show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1349))

    def recv_until(self,symb=b'\n>'):
        """Получаем сообщения с сервера, по умолчанию до приглашения"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self,show=True):
        """Получить параметры задания с сервера"""
```

```

data=self.recv_until()
try:
    data=data.decode()
except UnicodeDecodeError:
    print ('Error decoding unicode. Try connecting to server_
↪again.')
```

return (None,None)

```

if show:
    print (data)

p=int(re.search(r'(?<=GF\()\d+(?=\))',data).group(0))
group_order=int(re.search(r'(?<=order )\d+',data).group(0))
A=int(re.search(r'(?<=u\^3\+)(-?\d+u)',data).group(0)[: -1])
return (p,A,group_order)

def checkSolution(self,k, show=True):
    """Проверить решение"""
    self.s.sendall(('check '+str(k)+'\n').encode())
    data=self.recv_until(b'\n')
    try:
        data=data.decode()
    except UnicodeDecodeError:
        print ('Error decoding unicode. Try connecting to server_
↪again.')
```

return None

```

if show:
    print (data)
if data.find('flag')!=-1:
    return True
else:
    data=self.recv_until(b'>')
    try:
        data=data.decode()
    except UnicodeDecodeError:
        print ('Error decoding unicode. Try connecting to_
↪server again.')
```

return None

```

if show:
    print (data)
return False

def getPointMultiple(self,u, show=True):
    """Получить k*<ваша точка> с сервер"""
    self.s.sendall(('mult '+str(u)+'\n').encode())
    data=self.recv_until()
    try:
        data=data.decode()
```

```

        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server,
↪again.')
```

```

            return None
        if show:
            print (data)

        point=re.search(r'\d+',data).group(0)

        return int(point)

    def __del__(self):
        self.s.close()

try:
    from gmpy2 import invert
except ImportError:
    try:
        from Crypto.Util.number import inverse as invert
    except ImportError:
        print ("You need to install either gmpy2:\nsudo apt install,
↪python-gmpy2\nor pycryptodome:\npython3 -m pip install,
↪pycryptodome")
        raise Exception
def crt(remainders,modules,M):
    result = 0
    for (a, b) in zip(remainders,modules):
        result = (result+a*((M)//b)*invert((M)//b, b)) % M
    return result

```

```

[ ]: vs=VulnServerClient()
    p,A,group_order=vs.getChallenge()
    ladder=create_ladder(p,A)
    print (ladder(4,group_order),ladder(4,group_order-1))
    print (vs.getPointMultiple(4,show=False))

```

```

[ ]: vs.checkSolution(1)

```